

Generating High-Performance CUDA GEMM Kernels **with Reinforcement Learning**

A gate-driven pipeline for CUDA GEMM kernel generation, why the reward had to be a deterministic device-measured verifier rather than a learned one, why an offline preference stage (DPO) produced the promoted policy while an on-policy stage (GRPO) did not, and how a dtype-specific prompt contract and a repaired extraction layer turned a fragile synthesis task into a twice-reproduced pass.

40 / 40

CANDIDATES COMPILED,
RAN, AND WERE
CORRECT

*primary final evaluation,
reproduced on an
independent repeat run*

**6.56× /
16.86×**

MEDIAN / BEST SPEEDUP
VS. NAIVE

*primary run; repeat 6.57× /
17.10×, fp16 WMMA tensor-
core kernel at the high end*

**DPO
promoted
▪ GRPO
did not**

WHICH RL-FAMILY STAGE
PRODUCED THE POLICY

*single-update GRPO leaves
its clipped surrogate
inactive by construction*

Oluwaseyi Awoga · oawoga@engineering.upenn.edu · transformers · torch · peft
(LoRA) · Qwen2.5-Coder-3B · Modal A10 · nvcc 12.1 · cuBLAS oracle · July 2026

ABSTRACT

This paper reports a reinforcement learning style pipeline for generating correct, fast CUDA GEMM kernels from a code-pretrained language model. Unlike RLHF-style alignment or a learned security reward model, the reward here is a deterministic, device-measured verifier: a candidate kernel is compiled with a pinned `nvcc`, executed on a real GPU in a sandboxed subprocess, checked numerically against cuBLAS with a scale-normalized error metric, and scored by its profiled speedup over an in-house naive baseline through a roofline-attainment term that is gated to zero unless the kernel actually beats naive speed. Candidates may not call cuBLAS directly, define their own `main`, or rely on declarations that existed only in the prompt; cuBLAS is an oracle, never a candidate shortcut. The intended pipeline was two RL-family stages: Direct Preference Optimization (DPO) over optimized-versus-naive kernel pairs, followed by Group Relative Policy Optimization (GRPO) scored directly by the harness. DPO was mechanically healthy (training loss departed $\ln 2$ and finished below the 0.60 target), and, with a dtype-specific prompt contract (internally versioned `p04-v9`, the ninth and final iteration of the project's prompt-engineering process) and a corrected code-extraction layer, produced the model that passed final evaluation. GRPO, implemented here as a single on-policy update per sampled group (so the PPO/GRPO importance ratio is identically 1 and the clipped surrogate is inactive by construction, leaving REINFORCE with a group-relative baseline, a per-token KL penalty against a frozen reference, and an entropy bonus), surfaced two real failure modes, an early prompt that anchored the policy to naive-speed code and a reward-inflation artifact from pooling speedups across correct and incorrect candidates alike, but did not produce the model that was ultimately promoted.

On the primary final evaluation record (`p04_final_eval_20260705_172607`), the DPO-adapted policy under the final prompt contract generated 40 candidates across 10 GEMM shape/dtype specifications with a 100% compile rate, 100% correctness among compiled kernels, 100% shared-memory use among correct kernels, a $6.5566\times$ median speedup, and a $16.8606\times$ best speedup against naive, with zero failure buckets. An independent repeat run (`p04_final_eval_20260705_202234`) reproduced the pass at $6.5746\times$ median and $17.1033\times$ best speedup. The central methodological finding is that, in this domain, the training algorithm mattered less than the measurement and contract infrastructure around it: a correctness oracle that gates rather than averages, a naive-relative (not chip-peak-relative) speed target, a prompt that demonstrates a fast kernel family rather than only a compilable one, and an extraction layer that preserves the full CUDA preamble were jointly necessary, and any one of them missing produced a specific, diagnosable failure mode rather than a silent regression.

1 Introduction and background

1.1 Overview

A large and growing share of the CUDA that runs in production is now written by language models on request, rather than hand-tuned by an engineer who reasons directly about warp scheduling, shared-memory banking, and tensor-core fragment layouts. This independent research project asks whether reinforcement learning style adaptation can turn a general code-pretrained language model into a working CUDA GEMM kernel generator: not a model that merely emits plausible-looking CUDA, but one whose output reliably compiles, computes the numerically correct answer against a trusted reference, and runs measurably faster than a naive baseline on real hardware.

Accelerator-kernel generation is a harder verification problem than most tasks in the RLVR (reinforcement learning from verifiable rewards) family. A math or puzzle verifier is a five-line checker; a CUDA-kernel verifier has to compile untrusted code with a real toolchain, execute it inside a sandboxed process under a timeout, compare its numerical output to a reference at an appropriate tolerance, and profile its latency on a real device, all before a single reward number can be assigned. Getting any one of those steps wrong (an unstable correctness metric, an extraction bug that silently drops a `#define`, a reward that never actually requires beating naive speed) produces a policy that looks like it is learning while it is in fact drifting toward a degenerate solution that satisfies the letter of the reward and not the spirit of the task.

1.2 Objectives

The project pursued four goals, in order:

1. **Establish whether a small-to-mid-size code-pretrained model can emit compilable CUDA at all**, before spending GPU time on preference or policy optimization.
2. **Build a reward that gates on correctness and requires beating a naive baseline**, rather than a smooth chip-peak-relative attainment score that a merely-correct kernel can already satisfy.
3. **Run a two-stage RL pipeline (DPO, then GRPO)**, and report honestly which stage, if either, produced the model that passes a strict, device-measured final evaluation.
4. **Diagnose every failure mode encountered** (prompt anchoring, extraction bugs, reward-metric inflation, dtype-vocabulary leakage) rather than reporting only the configuration that worked.

1.3 The system

Component	Choice and rationale
Base model	<code>Qwen/Qwen2.5-Coder-3B</code> , selected after the 0.5B–1.5B class failed the zero-shot CUDA compile gate
Stage 1 (offline)	DPO: hand-rolled loss (no <code>trl</code> dependency), optimized kernel = chosen, naive kernel = rejected, LoRA policy against a frozen reference obtained by disabling the LoRA adapter on the same weights
Stage 2 (on-policy)	GRPO-style: group-relative advantage, no critic; implemented as one on-policy update per group, so the clipped importance-sampling ratio is identically 1 and the update reduces to REINFORCE with a group baseline, a per-token KL penalty, and an entropy bonus
Reward	Deterministic and device-measured: compile with a pinned <code>nvcc</code> , run in a sandboxed subprocess, correctness gated against cuBLAS at a dtype-specific scale-normalized tolerance, then roofline attainment shaped by speedup vs. an in-house naive baseline
Oracle	cuBLAS, used only as the numerical reference and a logged comparison speed; candidates are rejected if they call cuBLAS directly
Compute	Modal, ephemeral GPU jobs; final evaluation measured on NVIDIA A10; A100-40GB is the original design target
Discipline	A gate-driven loop (base-model gate, dataset audit, oracle calibration, smoke test, DPO, GRPO, final evaluation), a device-grounded final decision, and an independent repeat run for reproducibility

1.4 The task: conditional CUDA GEMM generation

Each input specification is a GEMM instance, `C = A @ B`, with `A` of shape `M × K`, `B` of shape `K × N`, `C` of shape `M × N`, row-major layout, and dtype `fp32` or `fp16`. The policy must emit CUDA source containing the required kernel contract, `extern "C" __global__ void gemm_kernel(...)`, plus a `launch_gemm` entry point the harness calls to invoke it. The candidate source may not define its own `main`, call cuBLAS, or rely on declarations that existed only in the prompt; the harness rejects these with explicit, stable failure reasons (`missing_gemm_kernel`, `missing_gemm_kernel_definition`, `miss-`

`ing_launch_gemm`, `missing_launch_gemm_definition`, `defines_main`, `uses_cublas`, and `nvcc_error:*` variants for compiler failures; see `src/harness.py::validate_contract` and `src/kernel_optimizer.py::normalize_fail_reason`). cuBLAS is an oracle, not an implementation strategy: it provides the numerical reference and a logged speed point, but a candidate that merely wraps a cuBLAS call is rejected outright, because that would reduce the task to writing a library wrapper rather than synthesizing a kernel.

The dataset is a thin-slice micro-benchmark: 5 shapes (`5123`, `10243`, `20483`, `4096×1024×1024`, `1024×4096×1024`) times 2 dtypes (fp32, fp16), for 10 pairs total, each pair consisting of an in-house naive reference and an in-house optimized (tiled) reference. The full "hundreds of pairs" effort described in the project design was deliberately gated behind this 10-pair slice passing its own audit first.

1.5 Scoring and metrics

Category	Metric	Definition	Target
Correctness	<code>scaled_linf_err</code>	<code>max_i</code> <code>abs(candidate_i -</code> <code>reference_i) /</code> <code>max_i</code> <code>abs(reference_i)</code> against cuBLAS	fp32 < 1e-5 (full credit); fp16/bf16 < 1e-2 (full credit)
Performance	Roofline attainment	achieved FLOP/s ÷ <code>min(peak_compute,</code> <code>peak_bandwidth ×</code> <code>arithmetic</code> <code>intensity)</code> for the exact GPU SKU	perf term of the reward; unbounded above 1.0
Performance	Speedup vs. naive	profiled kernel latency vs. the in-house naive kernel, same shape/dtype/device	median ≥ 5.0×, best-of-group ≥ 10.0× at final eval
Data audit	Optimized/naive pair validity	both sides compile, both numerically correct, optimized ≥ 2× faster than naive on-device	10/10 pairs required
Final gates	Compile / correctness / speed	see Results at a glance	all four must pass, and pass a repeat run

2 The one mapping that makes this an RL problem

The conceptual step that turns CUDA generation into an RL problem, rather than ordinary supervised code completion, is the same one every RLVR project of this kind makes: the **state** is the prompt plus the tokens generated so far, the **action** is the next token, the **policy** $\pi\theta$ is the language model, an **episode** is one full kernel generation, and the **reward** arrives only at the end of the episode, after the finished kernel has been compiled, executed, and measured. What distinguishes this project from an RLHF pipeline (a *learned* reward model on human preferences) and a security-RLVR pipeline (a learned Bradley-Terry security reward model) is that no reward model is ever trained here: the reward is entirely deterministic, computed by the harness from compiler output, a correctness comparison against cuBLAS, and a device profiler. Structurally this places the project closer to a pure RLVR task (a math or puzzle verifier) than to a preference-learning task, except that the verifier is far more expensive and far more failure-prone than a five-line checker, because it has to survive untrusted CUDA source reaching an actual compiler and an actual GPU.

3 What each model in the pipeline predicts

This project runs two RL-family stages against the same deterministic reward function, so the "which model does what" table looks different from either the RLHF pattern (a trained reward model plus a value head) or the security-RLVR pattern (a trained reward model plus an independent runnable arbiter). There is no reward model in this pipeline at all: the harness itself is both the training signal and the independent arbiter, because it produces the same compile/correctness/speed verdict whether it is called once for a final evaluation candidate or thousands of times inside a GRPO loop.

Model	Sees (input)	Outputs	What it predicts	Trained or frozen	Trained toward
Policy (actor) $\pi\theta$	prompt + tokens so far	a distribution over the next token	the best next CUDA token to emit right now	trained; the model being adapted (LoRA)	Stage 1: DPO log-sigmoid loss on optimized-vs-naive pairs. Stage 2: REINFORCE-style loss on the harness reward, with a KL penalty to the reference and an entropy bonus
Reference π_{ref}	prompt + tokens so far	next-token distribution	what the un-adapted base coder would say	frozen; obtained by disabling the LoRA adapter on the same weights (no second model copy)	none; anchors both the DPO preference gap and the Stage-2 KL leash
Harness / reward function	one finished, extracted CUDA source string	compile result, correctness gate, profiled speedup, a single scalar reward	whether this exact candidate is a valid, correct, faster-than-naive kernel	never trained; a deterministic program	none; it is the ground truth the policy is optimized against, not a fitted model

Note: unlike a typical RLHF pipeline or a security-RLVR pipeline, there is no row here for a reward model, because none is trained. The absence of a learned reward model removes one entire failure mode this project would otherwise have had to guard against (a reward model that a policy can game without becoming actually correct or fast), at the cost of a reward function that is orders of magnitude more expensive per call (a compile-and-profile round trip rather than a forward pass through a small scalar head).

4 Literature review

4.1 Three ways to get a reward into policy optimization

Aspect	RLHF (learned RM + PPO)	Security RLVR (learned RM + GRPO + independent arbiter)	This project (deterministic harness + DPO, then GRPO)
Reward source	learned RM on human preferences	learned Bradley-Terry RM on CVE-verified pairs	deterministic harness: compile + cuBLAS correctness gate + profiled speedup
Baseline	learned critic (value head)	group mean, no critic	Stage 1: reference log-ratio (DPO has no explicit baseline term); Stage 2: group mean, no critic
Ground-truth arbiter	none built in; relies on human eval	a second, non-optimized runnable exploit-test harness	the harness is the ground truth; there is nothing else to game around it
Reward-hacking risk	high; the proxy is learned	high; mitigated by the independent arbiter	low for the correctness gate (binary, deterministic); real for the <i>speed</i> term, which, pooled and unfiltered, was inflated by fast-but-incorrect candidates (fixed by tracking correct-only speedup separately, see Method)
Cost per reward evaluation	one forward pass through a scalar head	one forward pass through a scalar head, plus occasional runnable-test calls	one full compile + sandboxed execution + device-profiling round trip

This project sits at the deterministic-reward end of this spectrum, closer to a math or code RLVR verifier than to either RLHF or the learned-security-reward pipeline in this same series, but it pays for the removal of a learned reward model with a reward

function that is far more expensive and far more brittle to implementation bugs than a scalar head. The extraction bug documented in the Method section below (a pre-amble-stripping error) has no analogue in a learned-RM pipeline: a learned reward model cannot silently discard a `#define` before scoring, but a text-extraction step in front of a compiler can, and did.

4.2 Preference pairs and Direct Preference Optimization

DPO (Rafailov et al., 2023, NeurIPS, arXiv:2305.18290) shows that the RLHF objective has a closed-form optimal policy that can be substituted into the Bradley-Terry preference loss, producing a supervised classification loss on `(chosen, rejected)` pairs against a frozen reference, with no explicit reward model and no PPO loop. This project applies the identical mechanics to a different labeling source: `chosen` is an in-house optimized (tiled) GEMM kernel and `rejected` is an in-house naive GEMM kernel, for the same shape and dtype. This is a deliberately narrow, code-side reuse of DPO: the preference signal is not "a human preferred this response" but "this kernel is known to be faster," which makes the preference labels exact rather than noisy, at the cost of covering only the 10 audited shape/dtype pairs the project built by hand.

4.3 Policy-gradient RL, GRPO, and its variant family

The section below, "The road to policy optimization," derives the full policy-gradient-to-GRPO ladder; the concern here is where GRPO (Shao et al., 2024, arXiv: 2402.03300) sits relative to what came after it and, concretely, which parts of that later literature are and are not present in this project's Stage-2 implementation.

	GRPO (Shao et al., 2024)	DAPO (Yu et al., 2025)	Dr. GRPO (Liu et al., 2025)	This project's Stage 2
The clip	symmetric on the surrogate	decoupled, raised upper bound	same as GRPO	inactive by construction: one on-policy update per sampled group means the importance ratio is exactly 1 at update time, so there is nothing to clip

	GRPO (Shao et al., 2024)	DAPO (Yu et al., 2025)	Dr. GRPO (Liu et al., 2025)	This project's Stage 2
Advantage	group-relative, $(r - \text{mean}) / (\text{std} + \epsilon)$	group-relative, dynamic sampling drops all-equal groups	group-relative, no length/std divisors	group-relative, $(r - \text{mean}) / (\text{std} + \epsilon)$; an all-equal (including all-zero) group returns all-zero advantages and the update is skipped rather than dividing by ≈ 0
KL handling	leash to a frozen reference, typically a k3-style low-variance estimator	overlong-reward shaping	n/a (focuses on the advantage, not KL)	a direct per-token mean log-probability gap against the frozen reference, $(\text{per_token_logp} - \text{per_token_ref_logp}).\text{mean}()$; a simpler linear estimator, not the k3 form $\exp(\delta) - \delta - 1$
Extra terms	none beyond the clip and KL	dynamic sampling, token-level loss	drops length/std normalizers	an explicit entropy bonus $(\text{entropy_coeff} = 0.01)$ subtracted from the loss, on top of the KL penalty

This project's Stage 2 is therefore best described as GRPO's *advantage estimator* (a group-relative baseline with no learned critic) implemented as a single-update REINFORCE step with a KL penalty and an entropy bonus, rather than the full clipped-surrogate GRPO algorithm. With exactly one on-policy gradient step per sampled group and no re-sampling epochs, the PPO/GRPO importance ratio $\pi\theta/\pi\theta_{\text{old}}$ is identically 1 at the moment of the update, so the clip term has nothing to do. This is an explicit, documented implementation choice (the module docstring of `src/kernel_optimizer.py` states it outright), not an oversight, and it matters for how the GRPO results in this paper should be read: the honest comparison is not "did clipped GRPO out-perform DPO" but "did a group-relative, KL-constrained on-policy update out-perform the off-

line DPO stage," and the answer, reported without qualification in Results at a glance, is no.

4.4 Roofline modeling for the performance reward

The performance term of the reward is a roofline attainment score: achieved FLOP/s divided by `min(peak_compute, peak_bandwidth × arithmetic_intensity)` for the exact GPU SKU in use (Williams, Waterman, and Patterson, 2009, *Communications of the ACM* 52(4), 65–76). Roofline attainment is preferable to a flat chip-peak-relative score because it accounts for whether a given GEMM shape and dtype is compute-bound or bandwidth-bound before judging how close a kernel came to the achievable ceiling; a purely peak-relative score would unfairly penalize a bandwidth-bound shape for never approaching the compute peak. Attainment is deliberately never clipped at 1.0: because fp32 peaks in the SKU table are CUDA-core FMA peaks (`src/hardware.py`), a kernel that reaches for TF32 or tensor cores against an fp32 CUDA-core roofline could, correctly, report attainment above 1.0, and clipping that away would erase real signal. (In this project's runs the fp16 kernels are scored against the fp16 tensor-core ceiling, so observed attainment stays well below 1.0; the un-clipped design is a forward-looking invariant, not a description of a value seen here.)

4.5 Related work on RL and LLMs for GPU/accelerator code

Two threads of prior work bracket this project. AlphaTensor (Fawzi et al., 2022, *Nature* 610, 47–53) used deep reinforcement learning to discover new, provably correct matrix-multiplication algorithms with fewer scalar multiplications than the best previously known constructions, establishing that RL can find genuinely novel, verifiably correct linear-algebra algorithms rather than merely imitate them; this project does not attempt algorithm discovery at that level, and instead targets the more modest, systems-level question of whether a code-pretrained LLM can be adapted to *implement* known kernel-optimization idioms (shared-memory tiling, tensor-core WMMA) correctly and quickly for a fixed set of shapes. KernelBench (Ouyang, Guo, Arora, Zhang, Hu, Ré, and Mirhoseini, 2025, arXiv:2502.10517) benchmarks whether LLMs can translate PyTorch workloads into efficient GPU kernels across single-operator, fused-operator, and full-architecture tasks, and reports that even frontier reasoning models match a PyTorch baseline in fewer than 20% of cases out of the box. This project's scope is narrower than KernelBench (10 GEMM shape/dtype specs rather than 250 diverse workloads) but goes one step further along one axis: rather than evaluating a frozen model zero-shot, it asks whether preference and policy optimization can move a model that starts with a non-trivial but modest zero-shot compile rate (0.5417 for the selected 3B base model) up to a policy that reliably clears strict, device-measured compile, correctness, and speed gates.

4.6 Where this project sits

This project borrows DPO's preference-loss machinery from the RLHF literature and GRPO's group-relative-advantage idea from the RLVR literature, but applies both to a

reward that is neither a human preference nor a learned scalar model: a deterministic, expensive, compile-and-profile verifier. Its central empirical contribution is not a new RL algorithm but a system-level finding about where such a pipeline actually breaks, not (primarily) in the optimization mathematics, but in the prompt's implicit demonstration of what a "good" kernel looks like, in the text-extraction layer between the model's output and the compiler, and in whether the reward's speed term is measured against a baseline the final evaluation actually cares about (naive) rather than one that is merely smooth (chip-peak attainment alone).

5 The road to policy optimization: from the policy gradient

Notation used below: θ is the policy's parameters; τ is one trajectory (here, one generated kernel); $\pi\theta$ is the policy itself; π_{ref} is the frozen reference policy; π_{old} is a snapshot of the policy taken before the current optimization step; ρ (or ρ_t) is the importance ratio $\pi\theta/\pi_{\text{old}}$; ϵ is PPO's clip range; β is a KL penalty coefficient (PPO/GRPO) or the preference temperature (DPO); $V\phi$ is a learned value function with its own parameters ϕ ; ∇ is the gradient operator; $\sum_t$ sums over timesteps within a trajectory; and G is the group size sampled per prompt in GRPO.

5.1 The objective

All of policy-gradient RL maximizes expected return:

$$J(\theta) = E_{\{\tau \sim \pi\theta\}}[R(\tau)]$$

$J(\theta)$ is the expected return under the current policy, the quantity every policy-gradient method below tries to increase; τ is a trajectory (here, one generated CUDA kernel) sampled from $\pi\theta$, and $R(\tau)$ is that trajectory's reward, assigned only once the finished kernel has been compiled, checked, and profiled.

5.2 The policy gradient

The log-derivative identity turns the gradient of an expectation into an expectation of a gradient (Sutton, McAllester, Singh, and Mansour, 2000, NeurIPS), so the reward itself (a non-differentiable compile-and-profile round trip in this project) never needs to be differentiated:

$$\nabla J = E_{\tau}[(\sum_t \nabla \log \pi\theta(a_t | s_t)) * R(\tau)]$$

∇J is the gradient of that expected return with respect to θ , the direction training actually steps in; the inner $\sum_t \nabla \log \pi\theta(a_t | s_t)$ sums the gradient of the log-probability of each action a_t taken at each step t of the trajectory, and multiplying that sum by the whole trajectory's reward $R(\tau)$ is what lets a non-differentiable reward (a compile-

and-profile round trip here) still produce a usable gradient, since $R(\tau)$ never needs to be differentiated itself.

5.3 REINFORCE: the Monte-Carlo estimator

Estimate the expectation with samples and step (Williams, 1992):

$$L_REINFORCE = -(\log \pi\theta(\tau)) * R(\tau)$$

$L_REINFORCE$ is the per-trajectory loss whose gradient reduces to the ∇J expression above; $\log \pi\theta(\tau)$ is the trajectory's total log-probability under the current policy (the sum of $\log \pi\theta(a_t | s_t)$ over every step), and the minus sign turns "maximize expected return" into the "minimize this loss" form an optimizer expects. Unbiased, but high variance: the reward multiplies the whole trajectory's log-probability.

5.4 Add a baseline: same gradient, less noise

Subtracting any action-independent baseline b leaves the gradient unbiased, since $E[\nabla \log \pi\theta * b] = b * \nabla 1 = 0$:

$$L = -(\log \pi\theta(\tau)) * (R(\tau) - b)$$

The only change from $L_REINFORCE$ is that $R(\tau)$ is replaced by $(R(\tau) - b)$: the baseline b is subtracted from the reward before it scales the log-probability, which lowers variance without biasing the gradient, per the identity above.

5.5 Actor-critic: a learned baseline

Let a learned value network $V\phi(s)$ be the baseline, giving the advantage $A = R - V\phi(s)$: lower variance, slight bias, but one extra network to train and keep in sync. Here s is the state the value network is conditioned on, and A (the advantage) is what every later formula below actually uses in place of the raw baseline-subtracted return.

5.6 TRPO and PPO: don't step off the cliff

Large policy steps destroy the policy. TRPO (Schulman, Levine, Moritz, Jordan, and Abbeel, 2015, ICML, arXiv:1502.05477) maximizes a surrogate subject to a hard KL trust region; PPO (Schulman, Wolski, Dhariwal, Radford, and Klimov, 2017, arXiv:1707.06347) replaces the hard constraint with a cheap first-order clipped surrogate on the importance ratio $\rho_t = \pi\theta / \pi_{old}$:

$$L_PPO = -E[\min(\rho * A, \text{clip}(\rho, 1 - \epsilon, 1 + \epsilon) * A)]$$

ρ is the importance ratio $\pi\theta/\pi_{old}$ (how much more, or less, likely the current policy is to take the action the behavior policy actually took), A is the advantage from the act-

or-critic formula above, and ε bounds how far ρ can drift from 1 before the clip caps its contribution, so a single update cannot move the policy arbitrarily far from where the rollouts were actually collected.

5.7 GRPO: delete the critic

GRPO (Shao et al., 2024) removes the value network entirely. For a prompt it samples a group of G completions and uses the group's own reward mean and standard deviation to form the advantage:

$$A_i = (r_i - \text{mean}(r_{1..G})) / (\text{std}(r_{1..G}) + \varepsilon)$$

r_i is the i -th completion's reward within its sampled group of G completions, and A_i is that completion's advantage: how much better or worse it scored than the group's own average, scaled by the group's own spread, with ε in the denominator only to avoid dividing by a near-zero standard deviation when a group's rewards are nearly identical.

then applies the PPO clip on A_i plus a KL leash to a frozen reference:

$$L_{\text{GRPO}} = -E[\min(\rho * A, \text{clip}(\rho, 1-\varepsilon, 1+\varepsilon) * A)] + \beta * \text{KL}(\pi_\theta || \pi_{\text{ref}})$$

This is the PPO loss above with A_i substituted in for the advantage, plus one addition: a KL penalty term, weighted by β , that measures how far the current policy π_θ has drifted from the frozen reference π_{ref} , keeping the optimization from wandering too far from a known-reasonable starting point.

This project's Stage 2 uses exactly the advantage line above (`group_advantages`, unit-tested on CPU) but, as documented in the Literature review section above, performs a single on-policy update per group, so `pt = 1` at update time and the clip has nothing to do. The effective loss actually optimized is

$$L_{\text{stage2}} = -A_i * (\text{logp}_i / n_{\text{tokens}}) + \text{kl_beta} * \text{KL}_i - \text{entropy_coeff} * H_i$$

A_i is the same group-relative advantage defined above; logp_i is the completion's total log-probability under π_θ , divided by n_{tokens} so longer and shorter completions are weighted comparably rather than the loss scaling with length; KL_i is the mean per-token log-probability gap against the frozen reference (the actual KL leash, with `kl_beta` the literal code parameter for β above); and H_i is the mean per-token entropy of the current policy over the completion, subtracted (via `entropy_coeff`) to reward the policy for staying somewhat undecided rather than collapsing to a single output too quickly.

5.8 DPO: delete the RL loop entirely

DPO (Rafailov et al., 2023) observes that the RLHF objective's optimal policy has a closed form in terms of the reward, and that substituting that closed form into the

Bradley-Terry preference loss yields a loss with no reward model and no RL loop at all, just a classification loss on `(chosen, rejected)` pairs against a frozen reference:

```
L_DPO = -log sigmoid(  $\beta$  * ( (logp $\theta$ (chosen) - logpref(chosen))
                             - (logp $\theta$ (rejected) - logpref(rejected)) ) )
```

Each inner term $(\log p_{\theta}(x) - \log \text{pref}(x))$ compares how much more likely the current policy makes a completion x versus the frozen reference; the loss rewards a growing gap between that quantity for the chosen (optimized) kernel and the rejected (naive) kernel, and β acts as an inverse temperature controlling how sharply the loss penalizes the two gaps being close together.

This project's Stage 1 implements this loss directly (no `trl` dependency, roughly 20 lines), with `chosen` = the in-house optimized kernel and `rejected` = the in-house naive kernel for the same shape and dtype, and with the frozen reference obtained by disabling the LoRA adapter on the same underlying weights rather than instantiating a second full model copy.

5.9 The ladder at a glance

Rung	Fix it adds	Cost	Used in this project
REINFORCE	Monte-Carlo policy gradient	high variance	the core of Stage 2's single-update loss
+ baseline	subtract <code>b</code>	none (still unbiased)	group mean, via <code>group_advantages</code>
actor-critic	learned <code>Vϕ</code> baseline	+1 network	not used; no critic anywhere in this pipeline
PPO	clip the ratio; reuse batches	none	implemented in principle, inactive in practice (ratio = 1)
GRPO	group mean replaces <code>Vϕ</code>	needs <code>G</code> samples/prompt	the advantage estimator used in Stage 2
DPO	delete the RL loop; classification loss on pairs	needs paired preference data	Stage 1, and the stage that produced the promoted policy

6 Method: gates, reward design, and the prompt/extraction contract

6.1 Gate-driven experimental loop

The project used gates instead of a single monolithic training run, each one designed to catch a different failure mode before expensive GPU time was spent on the next stage:

Gate	Purpose	Outcome
Item 0	Validate one known-good kernel path and the correctness metric	Passed
Gate 0	Select a base model with a non-trivial zero-shot CUDA compile rate	3B fallback passed (0.5417)
Gate 1	Audit the 10-pair micro-dataset (both sides compile, both correct, optimized $\geq 2\times$ naive)	Passed, 10/10 verified pairs
Gate 2	Calibrate oracle and scorer behavior	Passed
Gate 3	Smoke-test the generate/compile/run/reward loop	Passed
Stage 1 (DPO)	Preference-train optimized-vs-naive pairs	Mechanically healthy: loss departed ln 2, below the 0.60 target
Stage 2 (GRPO)	On-device compile/profile/reward training	Diagnosed two real failure modes; did not produce the final promoted policy
Final evaluation	Sample 40 candidates across 10 specs and check all speed/correctness gates	Passed, and reproduced on an independent repeat run

6.2 Base-model selection

The 0.5B–1.5B model class did not clear a 30% zero-shot compile-rate threshold even after prompt iteration; across two successive Gate-0 matrices the best 1.5B result rose only from 0.1250 to 0.1667. The 3B fallback then cleared the gate cleanly. This validated the Gate 0 assumption before any GPU time was spent on preference or

policy optimization: reinforcement learning cannot easily amplify a behavior (emitting compilable CUDA) that is essentially absent in the base model.

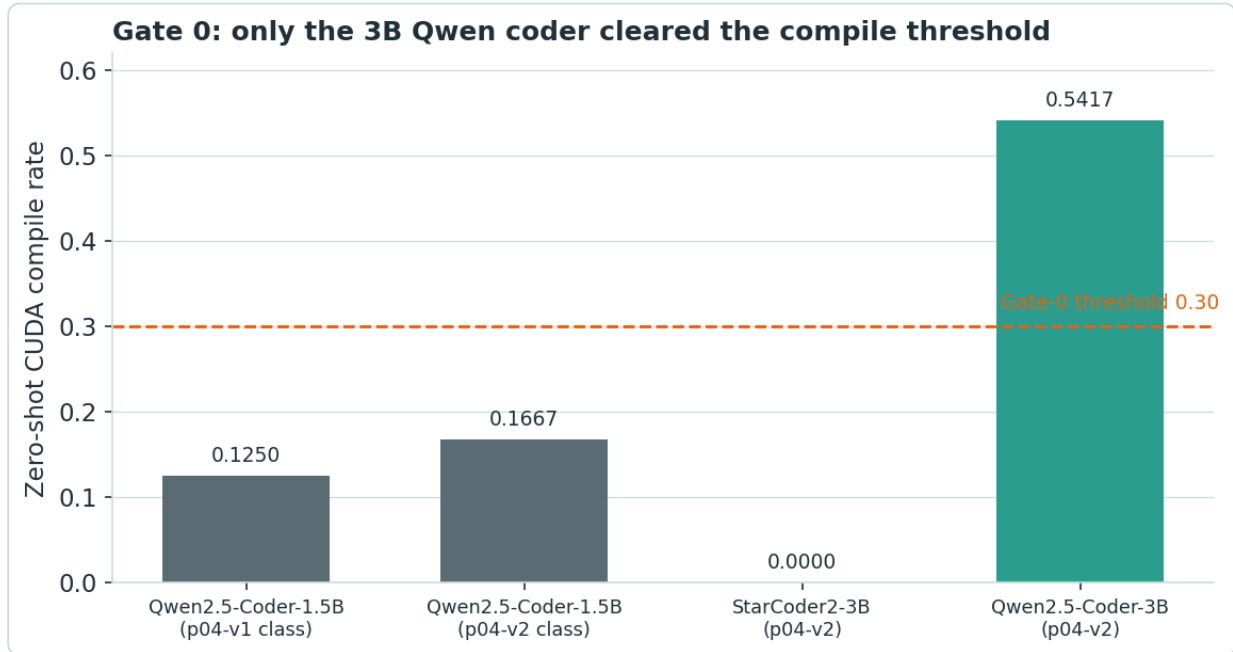


Figure 1: Gate 0: only the 3B Qwen coder cleared the zero-shot CUDA compile-rate threshold; StarCoder2-3B compiled nothing under the same prompt.

Prompt / candidate class	Best model	Compile rate	Result
Gate-0 matrix (0.5B-1.5B)	Qwen/Qwen2.5-Coder-1.5B	0.1250	Failed
Gate-0 matrix, iterated prompt (0.5B-1.5B)	Qwen/Qwen2.5-Coder-1.5B	0.1667	Failed
Gate-0 fallback (3B)	Qwen/Qwen2.5-Coder-3B	13/24 = 0.5417	Passed
Gate-0 fallback (3B)	bigcode/starcoder2-3b	0/24 = 0.0000	Failed

The selected base model is `Qwen/Qwen2.5-Coder-3B` (Hui et al., 2024, arXiv: 2409.12186), a code-specific model continued-pretrained from Qwen2.5 on more than 5.5 trillion tokens. (Records: `p04_gate0_compile_rate_matrix_20260704_182500`, `...184320`, and `p04_gate0_bigger_compile_rate_matrix_20260704_185949`.)

6.3 The correctness oracle

Correctness is measured against cuBLAS output, but the harness deliberately avoids raw elementwise maximum relative error as its primary metric: that quantity is unstable near zero-valued output entries, where legitimate K-term cancellation makes

the denominator tiny, so a provably correct naive GEMM can appear to have high relative error purely from test-matrix luck. The harness instead uses a scale-normalized error, `scaled_linf_err = max_i |candidate_i - reference_i| / max_i |reference_i|`, computed once over the whole output matrix rather than per element. The correctness gate is dtype-specific: fp32 requires `scaled_linf_err < 1e-5` for full credit (`< 1e-3` for a partial diagnostic tier); fp16/bf16 require `< 1e-2` for full credit (`< 5e-2` diagnostic). Correctness gates the reward outright: `RewardBreakdown` returns 0.0 whenever a kernel does not compile, does not run, or scores below full-credit correctness, regardless of how fast it appears to run. A wrong kernel at high throughput is not a partial success; it is a zero.

The oracle call itself (`src/harness.py`, the per-candidate compiled binary) is worth showing rather than only describing: cuBLAS is invoked once per timed run, immediately after a math-mode fix that the harness verifies rather than assumes took effect.

```
cublasHandle_t handle;
CUBLAS_CHECK(cublasCreate(&handle));
// Ampere+ cuBLAS defaults to TF32 tensor-core math even for "fp32" calls, which
// gives ~1e-3 relative error vs true fp32: fine for a speed baseline, not for a
// correctness oracle. Force strict IEEE fp32/fp16 accumulation so the oracle
// actually measures what its dtype claims.
CUBLAS_CHECK(cublasSetMathMode(handle, CUBLAS_PEDANTIC_MATH));
// Read the mode back rather than trusting the set call silently took: this is
// logged in the JSON output so a stale/uncompiled fix is detectable from the
// result itself, not assumed from source review.
cublasMath_t confirmed_math_mode;
CUBLAS_CHECK(cublasGetMathMode(handle, &confirmed_math_mode));

// cuBLAS reference, row-major trick: C_rm = A_rm @ B_rm <=>
// sgemm(col-major): C' (NxM) = B' (NxK) @ A' (KxM)
float alpha = 1.0f, beta = 0.0f;
#ifdef IS_FP16
CUBLAS_CHECK(cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
    dB, CUDA_R_16F, N, dA, CUDA_R_16F, K, &beta, dRef, CUDA_R_16F, N,
    CUBLAS_COMPUTE_32F, CUBLAS_GEMM_DEFAULT));
#else
CUBLAS_CHECK(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha,
    (const float*)dB, N, (const float*)dA, K, &beta, (float*)dRef, N));
#endif
```

The `cublasSetMathMode(CUBLAS_PEDANTIC_MATH)` line is a fix, not boilerplate: without it, cuBLAS silently uses TF32 tensor cores for "fp32" GEMM on Ampere-class GPUs, which would make the oracle itself imprecise relative to the true-fp32 candidates it is supposed to be checking. Forcing pedantic math and then reading `confirmed_math_mode` back means a future regression (someone removing this line) would show up in the logged JSON, not just in source review.

6.4 Performance reward: roofline attainment shaped by speedup vs. naive

The performance term is `attainment = achieved_flops_per_sec / roofline_flops(m, n, k, dtype, device)`, where `roofline_flops = min(peak_compute[dtype], peak_bandwidth × arithmetic_intensity)` for the exact GPU SKU (Williams et al., 2009). GPU peaks are keyed by the exact `torch.cuda.get_device_name()` string, and the code raises rather than guesses on an unknown SKU, a discipline adopted after an earlier related project conflated A10 and A10G peaks (they are distinct SKUs with different fp16/bf16 ceilings, 125 vs. 70 TFLOP/s dense, though identical fp32 peaks of 31.2 TFLOP/s and identical HBM bandwidth). Attainment alone, however, does not force improvement over the in-house naive baseline: a correct but naive-speed kernel could still earn non-zero, roofline-scaled reward, so GRPO had no explicit pressure to discover shared-memory tiling or tensor-core use. The reward was therefore patched to include speedup shaping against the naive baseline: a kernel at or below naive speed scores exactly 0 for the performance term (correctness is necessary but not sufficient); between 1.0× and 1.2× the multiplier ramps linearly to 1.0, avoiding a hard cliff exactly at 1.0× (which is within the run-to-run measurement noise of the naive baseline itself); above 1.2× the multiplier keeps growing via `log2`, unbounded, so a 5× kernel still scores meaningfully above a 2× kernel and a 10× kernel higher still.

The naive baseline every candidate is judged against (`src/harness.py`, `NAIVE_GEMM["fp32"]`) is a one-thread-per-output-element kernel with no shared memory and no tiling, the floor the whole reward shaping above is anchored to:

```
extern "C" __global__ void gemm_kernel(const float* A, const float* B, float* C,
                                     int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k) acc += A[row * K + k] * B[k * N + col];
        C[row * N + col] = acc;
    }
}

extern "C" void launch_gemm(const float* A, const float* B, float* C,
                           int M, int N, int K, cudaStream_t stream) {
    dim3 block(16, 16);
    dim3 grid((N + 15) / 16, (M + 15) / 16);
    gemm_kernel<<<grid, block, 0, stream>>>(A, B, C, M, N, K);
}
```

Every global memory access above is uncoalesced-adjacent at best and re-reads the same row of `A` and column of `B` from global memory once per output element, with no reuse; this is the deliberately weak floor a 5× median and 10× best speedup have to clear, not a strawman tuned to make the policy look good. The fp16 naive variant is identical except for `__half2float / __float2half` conversions around a float accumulator, so precision, not algorithm, is the only difference between the two dtypes' floors.

Final evaluation uses the project's own success criteria directly:

```
compile_rate >= 0.80
correctness_rate_compiled >= 0.95
median_speedup_vs_naive >= 5.0
best_speedup_vs_naive >= 10.0
```

6.5 Training stages

Stage 1 (DPO) trains the LoRA policy so that, for each of 8 of the 10 audited shape/dtype pairs (2 held out for the preference-gap diagnostic), the optimized (tiled) kernel is preferred over the naive kernel, using the frozen-adapter-off trick above as the reference. Training loss was mechanically healthy, departing $\ln 2$ and finishing below the 0.60 target within one epoch. The held-out, reference-adjusted preference-gap diagnostic was mixed across iterations (an early raw held-out gap was negative; a later, reference-adjusted diagnostic was positive); because this diagnostic alone was not decisive, the project relied on device-measured final evaluation, not the training-time preference gap, as the actual pass/fail signal.

Stage 2 (GRPO-style) samples a group of 4 candidates per spec, scores each with the harness, forms the group-relative advantage, and updates the policy with the single-step REINFORCE-plus-KL-plus-entropy loss described earlier in "The road to policy optimization." Guards include a stage gate (all policy parameters must be finite before Stage 2 starts), a zero-variance-group skip (an all-equal, e.g., all-zero-reward, group produces all-zero advantages, and the update is skipped rather than dividing by a near-zero standard deviation), a NaN-loss sentinel, and a CUDA out-of-memory sentinel that clears gradients and the CUDA cache before continuing rather than crashing the run. Some Stage-2 runs showed upward reward trends or high *pooled* speedup diagnostics: one run (`p04_grpo_stage2_20260705_111511`) reported a $16.57\times$ last-quarter pooled speedup, but source inspection and a *correct-only* speedup metric told a different story. `speedup_vs_naive` is computed for every candidate regardless of the correctness gate, so an unfiltered pooled mean can be dominated by a wrong-but-degenerate kernel that looks blazing fast rather than by a genuine tiled speedup; once the correct-only figure was tracked separately, no Stage-2 adapter matched what the DPO adapter under `p04-v9` achieved at final evaluation. The final passing policy used the Stage-1 (DPO) adapter, `checkpoints/dpo_adapter.tar.gz`, with the `p04-v9` prompt contract; no Stage-2 adapter was promoted as the final policy.

6.6 Prompt and extraction contract

The final prompt version, internally versioned `p04-v9` (the ninth and last iteration of the project's dtype-specific prompt-engineering process), encodes two different kernel families depending on dtype:

dtype	Prompted kernel family	Main idea
fp32	Stable wide-tiled GEMM	Shared-memory tiling, 64×64 output tile, 32-deep K tile, 4×4 register blocking
fp16	WMMA tensor-core GEMM	Warp-level matrix-multiply-accumulate via <code><mma.h></code> , with float accumulators stored through shared scratch before the half-precision write-back

This dtype split mattered because a single generic prompt caused the model to leak WMMA vocabulary into fp32 completions or to mutate macro names into invalid values; the immediately prior prompt version, `p04-v8`, already introduced dtype-specific references but left fp32 generations brittle for exactly this reason (fp32 candidates almost always mutated the tile macros into the runtime problem dimensions). `p04-v9` fixes this by using stable, descriptive fp32 macro names (`TILE_ROWS`, `TILE_COLS`, `TILE_DEPTH`, `THREAD_ROWS`, `THREAD_COLS`) and reserving WMMA-specific names (`WMMA_M`, `WMMA_N`, `WMMA_K`, `WARPS_M`, `WARPS_N`) strictly for fp16, with a dtype-specific self-containment warning so an fp32 prompt never even sees the WMMA names.

Concretely, `p04-v9` is not a single sentence of instruction but an assembled prompt with a grading preamble, a worked reference example, a dtype-specific self-containment warning, and a footer stating the exact function contract, with the reference example and the warning swapped in per dtype. The excerpt below is the actual grading preamble and the opening of the fp32 reference note, taken directly from `src/prompts.py`:

```
// You are writing one CUDA C++ source snippet for the evaluation harness.
// Output only CUDA C++ code: no markdown fences, no explanations, no main(),
// no cuBLAS calls, and no printf.
//
// GRADING: your kernel scores 0 unless it (a) compiles, (b) runs, and
// (c) matches a cuBLAS reference to tolerance. Among CORRECT kernels, one
// that only matches naive (one-thread-per-output, no shared memory) speed
// ALSO scores ~0 -- reward requires beating a naive baseline on the SAME
// shape, and keeps growing the faster you are past it.
//
// ===== REFERENCE EXAMPLE START (not part of your answer) =====
// A correct, faster fp32 __shared__-memory-tiled kernel. It reuses A/B
// tiles, uses a 4x4 register tile per thread, and avoids shared-memory bank
// conflicts. Copy this structure exactly: keep TILE_ROWS=64, TILE_COLS=64,
// TILE_DEPTH=32, THREAD_ROWS=4, THREAD_COLS=4, and block(16,16). These are
// literal tile constants; NEVER set them equal to M, N, K, or the requested
// problem shape. Do not invent TILE, do not change dimensions, and do not call
// cuBLAS.
```

followed by the actual `FP32_STABLE_WIDE_GEMM` reference kernel source (the worked example the model is shown), and a closing self-containment warning naming every macro and array the model must redeclare if it reuses the reference structure. The grading line ("a kernel that only matches naive speed ALSO scores ~0") is doing real work here: it is the plain-language restatement of the speedup-shaping reward described above, given directly to the policy at generation time, not only computed after the fact.

The extraction layer was equally critical. Earlier extraction logic began at the `extern "C" __global__` contract and, in doing so, accidentally stripped the preamble lines above it: `#define TILE 16`, `#include <mma.h>`, `using namespace nvccuda;`, causing widespread `nvcc_error:undefined_identifier` failures on code that was, in fact, correctly written by the model. The extractor was patched (extraction version `contract-with-preamble-v1`) to preserve contiguous CUDA preamble lines immediately before the kernel contract, and tests were added to protect the behavior.

7 Data

7.1 The 10-pair micro-dataset

The dataset is 5 shapes (`5123`, `10243`, `20483`, `4096×1024×1024`, `1024×4096×1024`) times 2 dtypes (fp32, fp16), for 10 pairs total, drawn from in-house naive and tiled reference templates (`src/dataset.py` pairs `NAIVE_GEMM` as rejected with `TILED_GEMM` as chosen). Gate 1's audit requires, per pair: both the naive and optimized source compile; both are numerically correct at the dtype-specific tolerance; and the optimized kernel is at least 2× faster than the naive kernel on the target device. All 10 pairs passed. The full "hundreds of pairs" target described in the project design remains gated behind this thin slice's own success, and was not pursued here.

7.2 Direct prompt reference audits

Before trusting the trained policy, the prompt's own reference kernels were audited directly on device (record `p04_prompt_reference_audit_20260705_171022`), separating "does the reference family work?" from "can the model reliably emit it?" The wide-tiled fp32 reference, audited across all 10 specs, was correct on 10/10 and reached a 5.3398× median speedup, but its best speedup was only 6.7921×, below the 10× best-of-group final-eval target, so the audit's `wide_reference_pass` flag was `false`, marking it a useful fp32 family but insufficient as the *only* kernel family in the prompt. Adding a corrected fp16 WMMA reference (float accumulators through shared scratch, a 2×2 fragment tile per warp) changed the ceiling: the dtype-specific reference audit passed (`prompt_reference_pass: true`) with 10/10 compiled, 10/10 ran, 10/10 correct, a 5.7368× median speedup, and a 13.4418× best speedup, justifying the dtype-specific `p04-v9` prompt design before the trained policy was ever evaluated against it.

8 Results at a glance

The DPO-adapted policy, evaluated under prompt `p04-v9`, passes all four final evaluation gates on the primary run and reproduces the pass on an independent repeat run, with no failure buckets in either run.

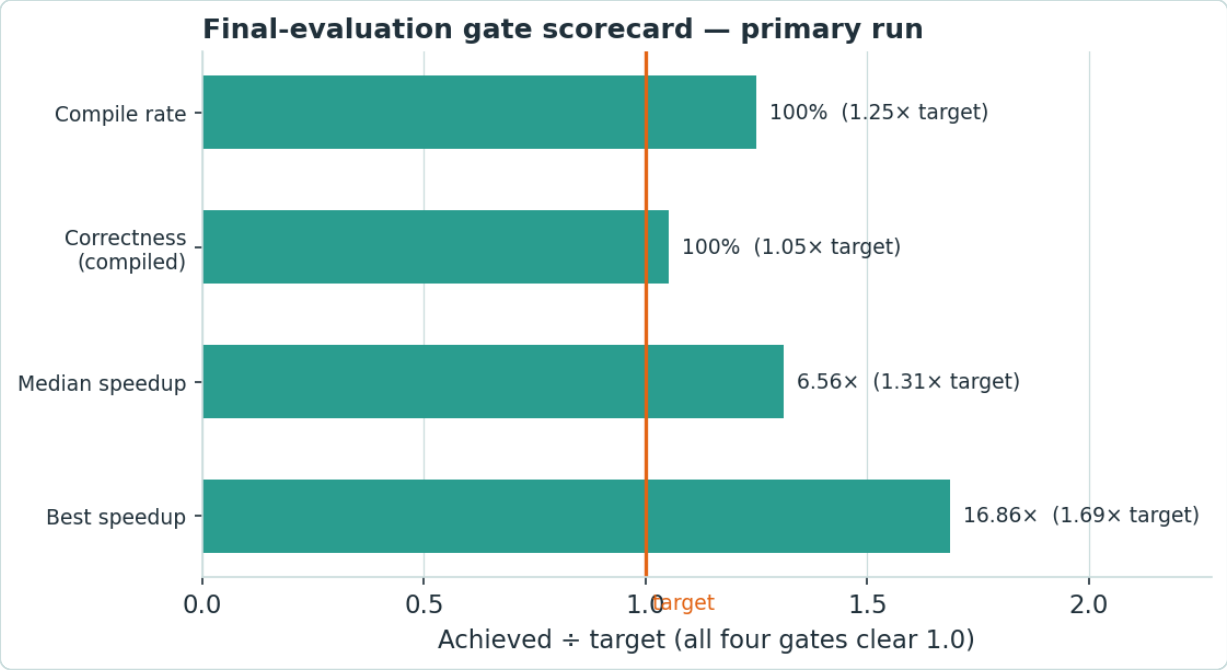


Figure 2: Final-evaluation gate scorecard: all four gates clear their target on the primary run.

Question	Answer
Can a small-to-mid code model emit CUDA zero-shot?	Only at 3B and above in this project; the 0.5B–1.5B class stayed below the 30% compile-rate threshold
Which stage produced the promoted policy?	DPO (Stage 1); GRPO (Stage 2) surfaced two real failure modes but did not promote
Did the final policy pass twice?	Yes: primary run <code>p04_final_eval_20260705_172607</code> and repeat run <code>p04_final_eval_20260705_202234</code> , both with zero failure buckets
Did fp16 or fp32 kernels achieve larger speedups?	fp16 (WMMA tensor-core), because the prompt induced tensor-core kernels and the fp16 roofline ceiling is roughly 4× the fp32 ceiling on the A10 (125 vs. 31.2 TFLOP/s dense)

9 Experiments and results

Each experiment states its objective, what informed it, the configuration for the significant finding, and the result.

9.1 E1: Gate 0, base-model zero-shot compile-rate probe

Objective. Decide whether reinforcement learning has anything to amplify before spending GPU time on DPO or GRPO. **Result.** The 0.5B–1.5B class stayed at 0.1250–0.1667 compile rate across two successive prompt matrices; `Qwen/Qwen2.5-Coder-3B` reached 0.5417 (13/24) in the fallback matrix, while `bigcode/starcoder2-3b` compiled nothing (0/24) under the same prompt. **Decision.** Escalate to the 3B class; select `Qwen/Qwen2.5-Coder-3B` as the base model.

9.2 E2: Gates 1–3, dataset audit, oracle calibration, and smoke test

Objective. Validate the data, the correctness oracle, and the end-to-end generate/compile/run/reward loop before any training. **Result.** All 10 dataset pairs passed the compile/correct/ $\geq 2\times$ -naive audit; the oracle and scorer behaved as designed on known-good and known-bad inputs; the smoke test exercised the full loop without incident. **Decision.** Proceed to Stage 1 (DPO).

9.3 E3: Stage 1, DPO on optimized-vs-naive pairs

Objective. Preference-train the policy to prefer optimized kernels over naive kernels for the same shape/dtype. **Config.** LoRA policy, `beta = 0.1`, `learning_rate = 5e-5`, `batch_size = 4`, 1 epoch, gradient clip 1.0, frozen reference via adapter-disable. **Result.** Training loss departed $\ln 2$ and finished below the 0.60 target; the held-out, reference-adjusted preference-gap diagnostic was mixed across iterations (negative in an early raw form, positive in a later reference-adjusted form). **Decision.** Treat DPO as mechanically viable, but do not treat the training-time preference gap as decisive; rely on device-measured final evaluation.

9.4 E4: Stage 2, the naive-anchoring failure

Objective. Test whether an early prompt version, which included a naive kernel as its main worked example, still let GRPO discover fast code. **What informed it.** In the early Gate-0-era prompt (`p04-v2`), the only worked example was a verbatim copy of the harness's own naive GEMM; the prompt was, in effect, demonstrating the exact kernel the project is trying to beat. **Result.** Final evaluation after this early GRPO run showed correct kernels with speedups near $1.0\times$: the model had learned the required contract, but also the naive implementation shape. This was a behavioral failure despite apparent training progress (compile rate and correctness were both fine). **Lesson.** In accelerator-code generation, a prompt's worked example is not neutral; if the

only concrete example is naive, the model may learn "compilable GEMM" rather than "fast GEMM." The fix (starting at `p04-v3`) relabeled the naive example as "do not submit this" and, over later revisions, replaced it with genuinely fast reference families.

9.5 E5: Stage 2, pooled-vs-correct-only speedup diagnostics

Objective. Determine why some GRPO runs showed promising aggregate speedup metrics that did not translate into a passing final evaluation. **What informed it.** `speedup_vs_naive` is computed for every candidate regardless of the correctness gate, so a pooled, unfiltered mean can be dominated by a kernel that is fast because it is wrong (it skips real work and fails the correctness check) rather than by a genuinely optimized kernel; run `p04_grpo_stage2_20260705_111511` showed a $16.57\times$ last-quarter *pooled* speedup that this artifact fully explains. **Fix.** Track `mean_speedup_vs_naive_correct_only` alongside the pooled `mean_speedup_vs_naive`, and use only the correct-only figure for any promotion decision. **Result.** Once this distinction was tracked, no Stage-2 adapter's correct-only speedup matched what the DPO adapter under `p04-v9` achieved at final evaluation.

9.6 E6: Prompt iteration, `p04-v8` to `p04-v9`, and the extraction-bug fix

Objective. Fix two compounding failures found while iterating the prompt: brittle fp32 generations under a dtype-mixed prompt (`p04-v8`), and an extraction bug that stripped required CUDA preambles. **What informed it.** An intermediate revision (`p04-v5`, with the repaired extractor) proved the model *could* emit shared-memory kernels again: final-eval compiles jumped from 0/40 to 25/40, correctness among compiled reached 0.96, and shared-memory use among correct reached 1.0, but `p04-v5`'s simple TILE-16 reference (`harness.SIMPLE_TILED_GEMM`) plateaued at roughly $1.3\text{--}1.6\times$ speedup, and GRPO did not invent register blocking from it (correct-only speedup actually fell from $1.24\times$ to $0.99\times$). In response, `p04-v6` returned to the stronger `harness.TILED_GEMM` reference. That result is what pushed the prompt toward stronger, dtype-specific reference families. **Result.** `p04-v8` introduced dtype-specific references but still leaked WMMA vocabulary into fp32 completions and let the model mutate macro names/dimensions; separately, contract-first extraction (starting at `extern "C" __global__`) dropped preamble lines like `#define TILE 16` and `#include <mma.h>`, producing `nvcc_error:undefined_identifier` failures on code that was otherwise correctly written. **Fix.** `p04-v9` strictly separates the fp32 and fp16 prompt surfaces with stable macro names; the extractor (`contract-with-preamble-v1`) preserves contiguous preamble lines immediately before the kernel contract, with tests added to protect the behavior.

9.7 E7: Prompt reference audits (informing the final prompt design)

Objective. Verify the prompt's own reference kernels work before trusting the trained policy against them (see the Data section above). **Result.** Wide-tiled fp32 reference (measured on NVIDIA A10): 10/10 correct, 5.3398× median / 6.7921× best speedup, insufficient alone against the 10× best-of-group target (`wide_reference_pass: false`). Corrected WMMA fp16 reference (measured on NVIDIA A10G, the same Modal `gpu="A10"` request's SKU-lottery counterpart): 10/10 compiled, ran, and correct, 5.7368× median / 13.4418× best speedup (`prompt_reference_pass: true`). Both audits report speedup against a same-device naive baseline, so the cross-device comparison doesn't invalidate either figure, but the two best-speedup numbers aren't directly comparable to each other given A10 and A10G's different fp16 ceilings. **Decision.** Justifies the dtype-specific `p04-v9` design as the final prompt contract.

9.8 E8: Final evaluation, primary run (the headline)

Objective. Evaluate the DPO-adapted policy under `p04-v9` against the project's own success criteria, sampling more candidates than any single training-time diagnostic. **Config.** 4 samples per spec × 10 specs = 40 candidates, temperature 0.2, max 2048 new tokens, NVIDIA A10, nvcc 12.1 (494.9 s wall, ≈12.4 s/candidate). **Result.**

Metric	Value	Target	Pass
Compile rate	1.0000	≥ 0.80	Yes
Correctness among compiled	1.0000	≥ 0.95	Yes
Median speedup vs. naive	6.5566×	≥ 5.0×	Yes
Best speedup vs. naive	16.8606×	≥ 10.0×	Yes
Shared-memory rate among correct	1.0000	diagnostic	n/a
Failure buckets	0	0 preferred	Yes

The best individual candidate (16.8606×, the `gemm_2048x2048x2048_fp16` spec, `p04_final_eval_20260705_172607.json`) is a WMMA tensor-core kernel, the fp16 family the `p04-v9` prompt targets. The eval log stores only a 700-character preview of its 3,050-character source (`source_snippet`, not logged in full), so what follows is that logged preview, not the complete kernel, cut off mid-statement exactly where the log itself is cut off:

```
#include <mma.h>
using namespace nvcuda;
```

```

#define WMMA_M 16
#define WMMA_N 16
#define WMMA_K 16
#define WARPS_M 4
#define WARPS_N 2
#define WARP_TILE_M 2
#define WARP_TILE_N 2
#define BLOCK_M (WARPS_M * WARP_TILE_M * WMMA_M)
#define BLOCK_N (WARPS_N * WARP_TILE_N * WMMA_N)
extern "C" __global__ void gemm_kernel(const __half* A, const __half* B, __half*
C,
                                int M, int N, int K) {
    int warp_id = threadIdx.y;           // block is dim3(32, 8): one warp per y
    int warp_m = warp_id / WARPS_N;
    int warp_n = warp_id - warp_m * WARPS_N;
    int row = blockIdx.y * BLOCK_M + warp_m * WARP_TILE_M * WMMA_M;
    int col = blockIdx.x * BLOCK_N + warp_n * WARP_TILE_N * WMMA_N;
    // [preview truncated at 700
    // of 3,050 characters]

```

Even this preview shows the shape of the win over the naive kernel above: one warp per WMMA tile ($\text{WARPS_M} \times \text{WARPS_N} = 4 \times 2 = 8$ warps per block, matching the `dim3(32, 8)` launch the comment documents) rather than one thread per output element, each warp cooperatively computing a $\text{WARP_TILE_M} \times \text{WARP_TILE_N}$ grid of 16x16 WMMA fragments instead of a single scalar accumulation. The full 3,050-character source (register/shared-memory staging of the WMMA fragments, the K-loop, and the half-precision write-back) is not recoverable from this log; logging complete kernel source for every candidate, not just a fixed-length preview, is a concrete, low-effort improvement for any follow-on run of this harness.

9.9 E9: Final evaluation, repeat run (reproducibility)

Objective. Confirm the pass is not a single lucky sample under the same adapter, prompt, and sampling configuration. **Result.** 40/40 compiled, ran, and correct again; median speedup 6.5746×, best speedup 17.1033× (501.2 s wall); zero failure buckets in both runs.

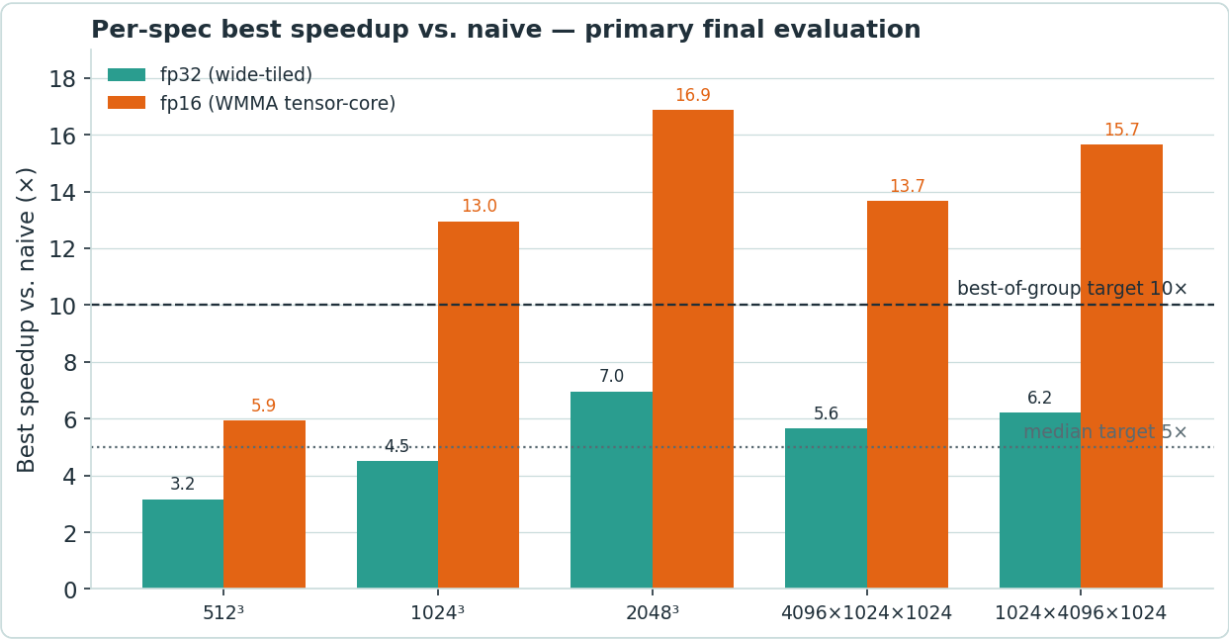


Figure 3: Per-spec best speedup vs. naive, primary final evaluation; fp16 WMMMA cases dominate the high end while fp32 wide-tiled cases carry the median.

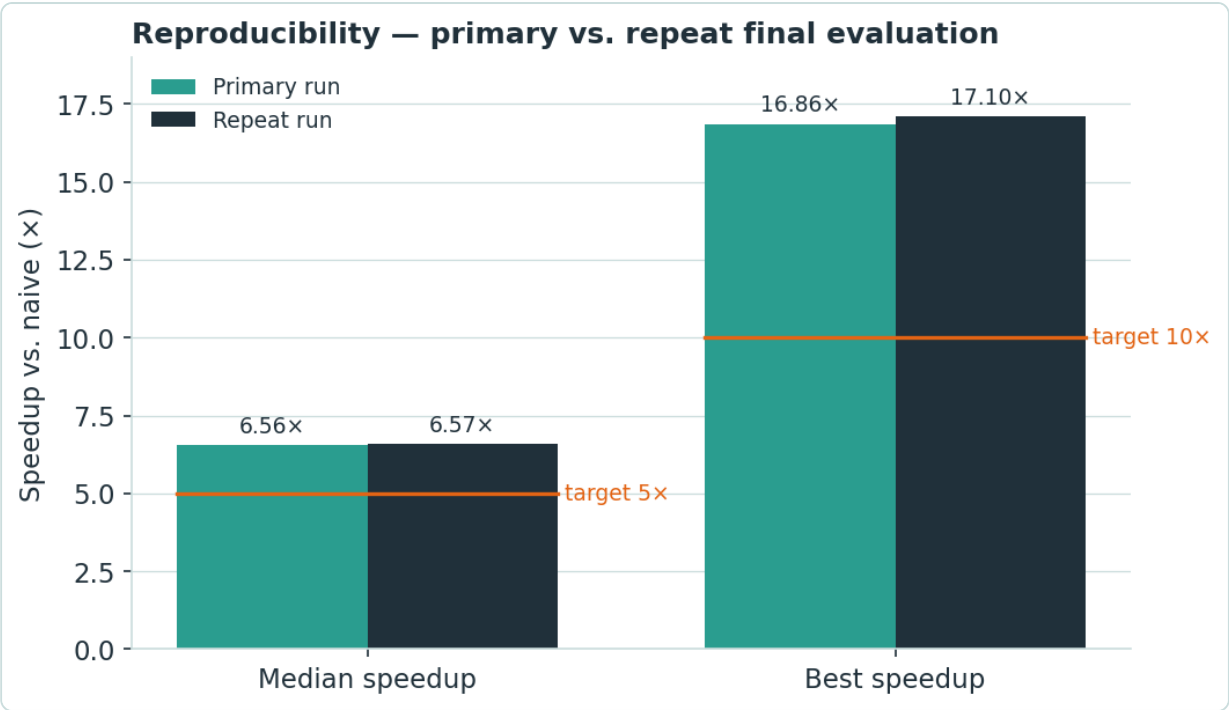


Figure 4: Primary vs. repeat final evaluation, aggregate median and best speedsups; both clear their targets with near-identical margins.

Metric	Primary eval	Repeat eval
Candidate count	40	40
Compile rate	1.0000	1.0000
Correctness among compiled	1.0000	1.0000

Metric	Primary eval	Repeat eval
Shared-memory rate among correct	1.0000	1.0000
Median speedup vs. naive	6.5566×	6.5746×
Best speedup vs. naive	16.8606×	17.1033×
Failure buckets	none	none

10 What worked and what did not

What worked.

- **A deterministic, gated reward.** Compile-then-correctness-then-speed gating (never averaging) meant a wrong kernel at high throughput could never be mistaken for progress.
- **Naive-relative speedup shaping.** Adding a hard zero below 1.0× naive speed, a linear ramp to 1.2×, and unbounded log-scaled growth above it gave GRPO (and the final evaluation gate) a target that actually matched what the project cared about, rather than a smooth chip-peak-relative score a merely-correct kernel could satisfy.
- **DPO on exact, code-side preference pairs.** Because "optimized beats naive" is a fact, not an opinion, DPO's preference loss had a clean, low-noise signal to train on, and it is the stage that produced the promoted policy.
- **The dtype-specific prompt contract.** Separating fp32 wide-tiling from fp16 WMMA vocabulary fixed a specific, diagnosed brittleness (`p04-v8`) rather than a vague "the model is inconsistent" complaint.
- **Correct-only speedup tracking.** Catching that pooled speedup metrics could be dominated by fast-but-wrong candidates prevented a false promotion.
- **The repeat run.** A same-configuration reproduction with zero failure buckets is meaningfully stronger evidence than a single 40-candidate pass.

What did not work.

- **The 0.5B-1.5B model class,** at any prompt version tried, for zero-shot CUDA compilation.
- **Attainment alone as the performance reward,** before speedup shaping was added: it let a correct-but-naive-speed kernel score non-zero, removing GRPO's pressure to discover tiling or tensor-core use.
- **A prompt whose only worked example was naive code:** it taught the contract but also taught the naive shape.
- **Contract-first code extraction:** it silently discarded correct preamble lines and produced spurious `undefined_identifier` compile failures.

- **GRPO as implemented here, for promotion:** it surfaced two real, useful failure modes (naive anchoring, pooled-speedup inflation) but never produced an adapter whose correct-only speedup matched the DPO adapter's final evaluation performance.

11 Limitations and challenges

1. **Only 10 GEMM shape/dtype specs.** The result does not establish broad CUDA generalization across arbitrary shapes, dtypes, or operation families beyond GEMM.
2. **A10 final GPU, A100-40GB design target.** Absolute timings, and possibly the relative ranking of fp32 vs. fp16 speedups, may differ on the original target hardware.
3. **The prompt contains strong reference kernel families.** The final behavior is partly prompt-conditioned synthesis (the model is shown a working tiled/WMMA structure) rather than pure RL discovery of a kernel family from scratch; the Method section's `p04-v9` design is explicit about this.
4. **Final evaluation uses 40 candidates.** More sampling could reveal residual stochastic failures, though the independent repeat run was clean.
5. **The naive baseline is intentionally simple.** Speedups are meaningful against the project's own naive reference, not against cuBLAS or a state-of-the-art hand-tuned library.
6. **GRPO, as implemented (single on-policy update per group), did not produce the final policy.** This project validates the overall gate-driven pipeline and the DPO stage, not GRPO's clipped surrogate specifically, since that surrogate was never actually active during training here.
7. **DPO's held-out preference-gap diagnostic was mixed across iterations.** Device-measured final evaluation, not this training-time diagnostic, was the actual pass/fail signal, and should be read as the primary evidence.
8. **The DPO reference is the same weights with the LoRA adapter disabled,** not an independently initialized second model; this is memory-efficient and exact, but ties the reference definition to the adapter mechanism rather than to a separately checkpointed base model.

12 Opportunities for extended study

- **Scale the audited dataset** beyond the 10-pair thin slice toward the "hundreds of pairs" target gated behind it, covering more shapes, non-square aspect ratios, and additional dtypes (bf16, int8).
- **Re-enable a genuine clipped-surrogate GRPO** (multiple sampled updates per group before re-sampling, so the importance ratio is not identically 1) and

compare it directly against the single-update REINFORCE-style Stage 2 reported here.

- **Extend beyond GEMM** to other accelerator primitives (reductions, convolutions, attention kernels) using the same gate-driven, naive-relative-speedup methodology.
- **Repeat the final evaluation on the A100-40GB design target** to close the A10-vs-A100 threat to validity, and to test whether the fp32/fp16 speedup ranking holds on a different roofline ceiling.
- **Investigate reward hacking more directly** for the speed term specifically, the one place where a learned-vs-deterministic distinction did not fully protect this project, since the *shaping* of a deterministic metric can still be gamed by pooling artifacts (as E5 showed), using a held-out, independent re-scoring arbiter, an independent runnable checker of the kind used in a security-RLVR pipeline to catch a policy that games the primary reward without actually being correct or fast.
- **Reduce reliance on strong prompt reference families** by testing whether a weaker, more schematic prompt combined with a longer GRPO budget can discover the same kernel families with less prompt-side scaffolding.

13 Methodological discipline (why the numbers are trustworthy)

This project treats reporting discipline as a first-class artifact, in the same spirit as related production-RL projects:

- **Gates in order.** Base-model zero-shot compile probe → dataset audit → oracle calibration → smoke test → DPO → GRPO → final evaluation. No later stage's claim is advanced on an earlier stage's training-time metric alone.
- **Device-grounded final decisions.** The mixed DPO held-out preference-gap diagnostic and the promising-looking pooled GRPO speedup metrics were both explicitly *not* trusted as promotion signals; only the device-measured final evaluation gates, computed by the same deterministic harness used throughout, decided promotion.
- **Correctness gates, never averages.** A wrong kernel scores exactly 0 regardless of its measured throughput; there is no configuration in which a fast, incorrect kernel can look like partial progress.
- **An independent repeat run.** The primary final evaluation record was reproduced under the identical adapter, prompt, and sampling configuration, with zero failure buckets in either run.
- **Honest negatives.** The 0.5B-1.5B compile-rate failure, the naive-anchoring failure, the pooled-speedup inflation, the extraction bug, and the fact that GRPO

did not produce the promoted policy are all reported as findings, not smoothed away or omitted.

14 Full configurations

14.1 Stage 1: DPO

```
policy: LoRA on Qwen/Qwen2.5-Coder-3B
reference: same weights, LoRA adapter disabled (no second model copy)
loss: -log_sigmoid(beta * ((logp_pol_chosen - logp_ref_chosen) -
(logp_pol_rejected - logp_ref_rejected)))
beta: 0.1 ; learning_rate: 5.0e-5 ; batch_size: 4 ; num_epochs: 1
gradient_clip: 1.0 ; min_tokens_per_side: 4 ; max_len: 2048
data: 8 of 10 audited optimized-vs-naive GEMM pairs (chosen = optimized, rejected
= naive); 2 held out for the preference-gap diagnostic
success criteria: train loss departs ln2 and < 0.60; held-out reference-adjusted
preference gap > 0 (mixed; not decisive alone)
```

14.2 Stage 2: GRPO-style (single on-policy update per group)

```
group_size: 4 ; max_new_tokens: 1536 ; temperature: 0.8
kl_beta: 0.1 ; entropy_coeff: 0.01 ; learning_rate: 5.0e-5 ; num_steps: 100
gradient_clip: 1.0 ; entropy_chunk_tokens: 32
advantage: (r - mean) / (std + 1e-6); all-equal group -> zero advantage -> update
skipped
loss: -(advantage * mean_completion_logp) + kl_beta * KL - entropy_coeff * entropy
KL: mean per-token log-prob gap vs. frozen reference (not a k3 estimator)
note: importance ratio == 1 at update time (one on-policy update per group); PP0/
GRPO clip inactive by construction
guards: stage-gate (finite params), zero-variance-group skip, NaN-loss sentinel,
CUDA-OOM sentinel
```

14.3 Final evaluation

```
model: Qwen/Qwen2.5-Coder-3B
adapter: checkpoints/dpo_adapter.tar.gz # Stage 1 (DPO); no Stage-2 adapter
promoted
prompt: p04-v9
samples_per_spec: 4 ; specs: 10 ; candidates: 40
temperature: 0.2 ; max_new_tokens: 2048
gpu: NVIDIA A10 ; nvcc: 12.1
success_criteria:
  compile_rate: ">= 0.80"
  correctness_rate_compiled: ">= 0.95"
  median_speedup_vs_naive: ">= 5.0"
  best_speedup_vs_naive: ">= 10.0"
```

14.4 Reward

```
correctness_gate: scaled_linf_err vs cuBLAS, dtype-tiered
  fp32: full_credit < 1.0e-5 ; partial_diagnostic < 1.0e-3
  fp16_bf16: full_credit < 1.0e-2 ; partial_diagnostic < 5.0e-2
performance_term: roofline_attainment = achieved_flops_per_sec / min(peak_compute,
peak_bandwidth * arithmetic_intensity)
speedup_shaping:
  zero_floor: 1.0x naive   # kernel at or below naive speed scores 0 for the perf
term
  full_credit_at: 1.2x naive # linear ramp 1.0x -> 1.2x
  above_1.2x: unbounded log2 growth, never clipped
reward: 0.0 if not compiled/ran/correct, else roofline_attainment *
speedup_shaping(speedup_vs_naive)
```

15 Hardware and installation

15.1 Hardware

- **Local (CPU).** Dataset build, extraction/reward/prompt unit tests, orchestration; no local GPU required.
- **GPU (Modal).** NVIDIA A10 (final evaluation, primary and repeat runs), with A10G observed in some training/audit runs; NVIDIA A100-40GB is the original design target, not yet re-run. Ephemeral, fast-scaledown jobs.
- **GPU peak table.** Keyed by the exact `torch.cuda.get_device_name()` string; raises on an unknown SKU rather than guessing (A10 and A10G are distinct SKUs with different fp16/bf16 peaks, 125 vs. 70 TFLOP/s dense, despite identical fp32 peaks of 31.2 TFLOP/s and identical HBM bandwidth of 600 GB/s).

15.2 Installation

```
cd Production_Grade/04_Performance_RL_Accelerators_WORKING_COPY
.venv/bin/python -m pytest tests/ -q           # local regression suite (last: 65
passed, 4 skipped)

# Modal (GPU)
pip install modal && modal setup

# final evaluation (the passing configuration)
/Users/oluwaseyiawoga/Library/Python/3.14/bin/modal run scripts/
run_final_eval_modal.py \
  --adapter-path checkpoints/dpo_adapter.tar.gz \
  --samples-per-spec 4 \
  --temperature 0.2 \
  --max-new-tokens 2048
```

16 Datasets and sources

Artifact	Identifier / path
Base model	<code>Qwen/Qwen2.5-Coder-3B</code> (Hui et al., 2024, arXiv:2409.12186)
Numerical oracle	cuBLAS (NVIDIA CUDA Toolkit 12.1)
Design document	<code>PATTERN_04_DESIGN.md</code>
Config	<code>config/pattern04.yaml</code>
Final report	<code>reports/performance_report.md</code>
Primary final-eval payload	<code>reports/final_eval_p04_final_eval_20260705_172607.json</code>
Repeat final-eval payload	<code>reports/final_eval_p04_final_eval_20260705_202234.json</code>
Gate-0 / audit records	<code>experiments/experiments.json</code>
Final adapter	<code>checkpoints/dpo_adapter.tar.gz</code>
Prompt implementation	<code>src/prompts.py</code>
Harness	<code>src/harness.py</code>
Reward	<code>src/reward.py</code>
Roofline / GPU specs	<code>src/hardware.py</code>
DPO trainer	<code>src/dpo_trainer.py</code>
GRPO-style trainer	<code>src/kernel_optimizer.py</code>
Dataset build/audit	<code>src/dataset.py</code>
Final-eval runner	<code>src/final_eval.py</code> , <code>scripts/run_final_eval_modal.py</code>
Test suite	<code>tests/test_core.py</code> (65 passed, 4 skipped, last local run)

17 Glossary

Term	Definition
RLVR	Reinforcement Learning from Verifiable Rewards: RL where a program, not a human or a learned model, scores outputs.

Term	Definition
GEMM	General Matrix Multiply, <code>C = A @ B</code> ; the target operation family for this project.
WMMA	Warp Matrix Multiply-Accumulate, the CUDA API for tensor-core fragment-based matrix math.
Roofline model	A performance model giving the attainable FLOP/s as <code>min(peak_compute, peak_bandwidth × arithmetic_intensity)</code> (Williams et al., 2009).
Arithmetic intensity	FLOPs performed per byte moved; determines whether a kernel is compute-bound or bandwidth-bound.
<code>scaled_linf_err</code>	<code>max_i abs(candidate_i - reference_i) / max_i abs(reference_i)</code> , the scale-normalized correctness metric used against cuBLAS.
DPO	Direct Preference Optimization; trains a policy from <code>(chosen, rejected)</code> pairs against a frozen reference with no explicit reward model or RL loop.
GRPO	Group Relative Policy Optimization; PPO without a value network, using the group mean as the baseline.
Bradley-Terry model	A model of paired comparisons underlying both DPO's and learned reward models' preference losses.
Speedup shaping	The reward's naive-relative multiplier: zero at or below naive speed, linear ramp to 1.2×, unbounded log2 growth above it.
Contract-with-preamble extraction	The fixed extraction version that preserves CUDA preamble lines (<code>#define</code> , <code>#include</code> , <code>using namespace</code>) immediately before the kernel contract.

18 Conclusion

This project reached its final evaluation target twice: the DPO-adapted policy, evaluated under the dtype-specific `p04-v9` prompt contract, generated CUDA GEMM kernels that compiled, ran, passed cuBLAS correctness checks, used shared memory, and exceeded the required speedup thresholds against an in-house naive baseline, on

both a primary evaluation and an independent repeat run, with zero failure buckets in either. GRPO, implemented here as a single on-policy update per sampled group (and therefore, by construction, without an active clipped importance-sampling surrogate), surfaced two genuine and useful failure modes (prompt-driven naive anchoring and pooled-speedup inflation) but did not produce the promoted policy. That distinction is reported plainly rather than implied away by the project's use of "reinforcement learning" in its framing, since DPO is itself an RL-derived, preference-based method and is the stage that actually worked. The strongest methodological lesson is that, in this domain, the measurement and contract infrastructure around the training algorithm (a correctness oracle that gates rather than averages, a speed target defined against the baseline the project actually cares about rather than one that is merely smooth, a prompt that demonstrates a fast kernel family rather than only a compilable one, and an extraction layer that preserves the full CUDA preamble) mattered as much as, and arguably more than, the choice between DPO and GRPO itself. For future work, the next step is not simply to run more GRPO with the current single-update implementation; the stronger directions are to re-enable a genuine clipped-surrogate GRPO for a fair head-to-head against DPO, to expand the audited dataset beyond 10 pairs, to extend past GEMM to other accelerator primitives, and to repeat the final evaluation on the A100-40GB design target.

19 References

1. Dandekar, Rajat. *Reinforcement Learning in Production*. Vizvara AI Labs, 2026.
2. Sutton, McAllester, Singh, Mansour. *Policy Gradient Methods for Reinforcement Learning with Function Approximation*. NeurIPS 2000.
3. Williams. *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning (REINFORCE)*. Machine Learning, 1992.
4. Schulman, Levine, Moritz, Jordan, Abbeel. *Trust Region Policy Optimization*. ICML 2015. arXiv:1502.05477.
5. Schulman, Wolski, Dhariwal, Radford, Klimov. *Proximal Policy Optimization Algorithms*. arXiv:1707.06347, 2017.
6. Shao, Wang, Zhu, Xu, Song, Bi, Zhang, Zhang, Li, Wu, Guo. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (GRPO)*. arXiv:2402.03300, 2024.
7. Rafailov, Sharma, Mitchell, Manning, Ermon, Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS 2023. arXiv:2305.18290.
8. Yu et al. *DAPO: An Open-Source LLM Reinforcement Learning System at Scale* (ByteDance), 2025. arXiv:2503.14476.
9. Liu et al. *Understanding R1-Zero-Like Training: A Critical Perspective* (Dr. GRPO; Sea AI Lab), 2025. arXiv:2503.20783.

10. Williams, Waterman, Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 52(4), 65–76, 2009.
11. Fawzi, Balog, Huang, Hubert, Romera-Paredes, Barekatin, Novikov, R. Ruiz, Schrittwieser, Swirszcz, Silver, Hassabis, Kohli. *Discovering Faster Matrix Multiplication Algorithms with Reinforcement Learning* (AlphaTensor). Nature, 610, 47–53, 2022. DOI 10.1038/s41586-022-05172-4.
12. Ouyang, Guo, Arora, Zhang, Hu, Ré, Mirhoseini. *KernelBench: Can LLMs Write Efficient GPU Kernels?* arXiv:2502.10517, 2025.
13. Hui, Yang, Cui, Yang, Liu, Zhang, Yu, et al. *Qwen2.5-Coder Technical Report*. arXiv:2409.12186, 2024.
14. `PATTERN_04_DESIGN.md`, the project's design document and success criteria.
15. `config/pattern04.yaml`, final configuration and selected-policy record.
16. `reports/performance_report.md`, final human-readable evaluation report.
17. `reports/final_eval_p04_final_eval_20260705_172607.json` and `reports/final_eval_p04_final_eval_20260705_202234.json`, primary and repeat machine-readable final evaluation payloads.
18. `src/prompts.py`, `src/harness.py`, `src/reward.py`, `src/hardware.py`, `src/dpo_trainer.py`, `src/kernel_optimizer.py`, `src/dataset.py`, `src/final_eval.py`, `scripts/run_final_eval_modal.py`.